

مستندات فایل tfs.py

تبدیلات داده برای سگمنتیشن ضایعات MS در چارچوب MS-SAM-LESS

پایان نامه کارشناسی ارشد – مهندسی کامپیوتر، گرایش هوش مصنوعی

۶ مهر ۱۴۰۵

فهرست مطالب

۲	۱	مقدمه
۲	۲	بخش ۱: وارد کردن کتابخانه‌ها (Imports)
۲	۱.۲	توضیح imports
۲	۳	بخش ۲: کلاس ResizeWithRatio
۲	۱.۳	سازنده (<code>__init__</code>)
۲	۲.۳	متد <code>__call__</code>
۴	۴	بخش ۳: تابع get_mslesseg_transform
۵	۱.۴	بخش ۱-۳: <code>transform_train</code>
۶	۲.۴	بخش ۲-۳: <code>transform_val_test</code>
۷	۱.۲.۴	بررسی <code>TransposeD(indices=(0, 3, 1, 2))</code>
۷	۵	مقایسه‌ی train و val/test
۷	۶	ارتباط با پروژه‌ی Polyp
۸	۷	جمع بندی

۱ مقدمه

فایل `tfs.py` مسئول تعریف تبدیلات داده (Data Transforms) برای دیتاست MSLesSeg است. این تبدیلات، حجم‌های MRI سه‌بعدی چندم‌دالیتی را برای ورود به مدل SAM-based آماده می‌کنند. هدف کلی این فایل، تولید دو pipeline مجزا است:

- **pipeline آموزش (train):** شامل augmentation قوی برای جلوگیری از بیش‌برازش.
- **pipeline ارزیابی (val/test):** بدون augmentation و با پردازش قطعی.

پسوند D در نام:ها: `transforms` در کتابخانه MONAI، پسوند D به معنای Dictionary-based است. این `transforms` روی یک دیکشنری کار می‌کنند و هر کدام روی کلیدهای مشخصی اعمال می‌شوند. ورودی معمولاً چیزی شبیه `{"image": ..., "mask": ...}` است.

۲ بخش ۱: وارد کردن کتابخانه‌ها (Imports)

```
1 from monai.transforms import MapTransform
2 import copy
3 from monai.transforms import (
4     Compose, LoadImageD, EnsureTyped, ResizeD, ResizeWithPadOrCropD,
5     OrientationD, RandFlipD, RandRotate90D, RandCropByPosNegLabelD,
6     EnsureChannelFirstD, NormalizeIntensityD, ScaleIntensityD,
7     SpacingD, Transposed
8 )
```

۱.۲ توضیح imports

۳ بخش ۲: کلاس ResizeWithRatio

۱.۳ سازنده (__init__)

```
1 class ResizeWithRatio(MapTransform):
2     def __init__(self, keys, image_size, mode='bilinear'):
3         super().__init__(keys)
4         self.image_size = image_size
5         self.mode = mode
```

توضیح:

- کلاس از `MapTransform` ارث‌بری می‌کند.
- `keys`: کلیدهایی که `transform` روی آن‌ها اعمال می‌شود (معمولاً `["image", "mask"]`).
- `image_size`: اندازه‌ی هدف (مثلاً ۱۰۲۴ برای SAM).
- `mode`: درون‌یابی — برای تصویر `bilinear` یا `trilinear` و برای ماسک `nearest`.

۲.۳ متد __call__

```
1 def __call__(self, data):
2     d = copy.deepcopy(data)
3     image = d[self.keys[0]]
4     h, w = image.shape[1:3]
```

توضیح:

جدول ۱: کاربرد هر import در این فایل

کاربرد	import
کلاس پایه برای ساخت transform سفارشی روی دیکشنری	MapTransform
برای کپی عمیق داده‌ها بدون تغییر اصلی	copy
زنجیره‌کردن چند transform پشت سر هم	Compose
بارگذاری تصویر از فایل (NIfTI/PNG) به صورت دیکشنری	LoadImageD
افزودن بُعد کانال در ابتدا (اگر نباشد)	EnsureChannelFirstD
تبدیل به tensor و انتقال به device	EnsureTyped
تغییر اندازه به ابعاد مشخص	Resized
تغییر اندازه با pad یا crop به ابعاد دقیق	ResizeWithPadOrCropD
تغییر جهت‌گیری آناتومیک تصویر	OrientationD
قرینه‌کردن تصادفی روی یک محور	RandFlipD
چرخش تصادفی ۹۰ درجه	RandRotate90D
برش تصادفی با نسبت مشخص از background و foreground	RandCropByPosNegLabelD
نرمال‌سازی z-score	NormalizeIntensityD
مقیاس‌دهی به بازه‌ی [۰, ۱]	ScaleIntensityD
resample به فاصله‌ی فیزیکی مشخص (میلی‌متر)	SpacingD
جاب‌جایی محورها	TransposeD

- data یک دیکشنری است.
- کپی عمیق می‌گیریم تا داده‌ی اصلی تغییر نکند.
- image را از اولین کلید (args['image_key']) استخراج می‌کنیم.
- `h, w = image.shape[1:3]`: چون بُعد کانال در ایندکس ۰ است، ابعاد فضایی از ایندکس ۱ به بعد شروع می‌شوند.

مثال: اگر `image.shape = (1, 512, 256, 30)` باشد، آنگاه `h = 512` و `w = 256` خواهد بود.

```

۱ if h != w:
۲     if h > w:
۳         new_h = self.image_size
۴         scale_factor = new_h / h
۵         new_w = int(w * scale_factor)
۶     else:
۷         new_w = self.image_size
۸         scale_factor = new_w / w
۹         new_h = int(h * scale_factor)
۱۰    new_size = (new_h, new_w) if image.ndim == 3 else (new_h, new_w,
        image.shape[3])
۱۱ else:
۱۲    new_size = (self.image_size, self.image_size) if image.ndim == 3
        else (self.image_size, self.image_size, image.shape[3])

```

منطق حفظ نسبت ابعاد:

- اگر ارتفاع و عرض برابر نیستند، نسبت ابعاد را حفظ می‌کنیم.
- ضلع بزرگ‌تر را به image_size می‌رسانیم و ضلع دیگر را با همان نسبت مقیاس می‌دهیم.

- اگر تصویر 2D باشد (C, H, W :ndim=3) $\rightarrow$ `.new_size = (new_h, new_w)`
- اگر تصویر 3D باشد (C, H, W, D :ndim=4) $\rightarrow$ بُعد سوم (D) دست نخورده باقی می ماند.

مثال‌ها:

- مثال ۱ $\rightarrow$ `.new_size = (1024, 512)` `image_size=1024`, `shape = (1, 512, 256)` (D):۲
- مثال ۲ $\rightarrow$ `.new_size = (1024, 512, 30)` `image_size=1024`, `shape = (1, 512, 256, 30)` (D):۳
- مثال ۳ (مربع): `.new_size = (1024, 1024, 30)` $\rightarrow$ `shape = (1, 512, 512, 30)`

```

1  resize_transform = ResizeD(keys=self.keys, spatial_size=new_size, mode
2  =self.mode)
3  d = resize_transform(d)
   return d

```

توضیح:

- یک `ResizeD` با ابعاد محاسبه شده می سازیم.
- آن را روی دیکشنری `d` اعمال می کنیم.
- نتیجه برگردانده می شود.

نکته‌ی مهم: این کلاس فقط به بُعد اول تصویر (`self.keys[0]`) نگاه می کند تا ابعاد را محاسبه کند، ولی `ResizeD` را روی همه‌ی کلیدها (شامل ماسک) اعمال می کند. این یعنی ماسک هم به همان ابعاد `resize` می شود که درست است.

۴ بخش ۳: تابع `get_mslesseg_transform`

```

1  def get_mslesseg_transform(args, sam_image_dim=None, get_bbox=False):
2  KEYS = [args['image_key'], args['mask_key']]

```

توضیح:

- این تابع دو `transform` برمی گرداند: یکی برای `train` و یکی برای `val/test`.
- `args`: دیکشنری آرگومان‌های برنامه؛ برای نمونه:
 - `image_key='flair'` و `mask_key='mask'`
 - `axcodes='RAS'` و `num_slices=30`
 - `device='cuda'`
- `sam_image_dim`: اندازه‌ی ورودی `SAM` (معمولاً ۱۰۲۴).
- `get_bbox`: (در این کد استفاده نشده) احتمالاً برای حالت `bounding-box prompt`.
- `KEYS`: کلیدهای تصویر و ماسک.

۱.۴ بخش ۳-۱: transform_train

```

1 transform_train = [
2     LoadImageD(KEYS),
3     EnsureChannelFirstD(keys=KEYS),
4     NormalizeIntensityD(keys=(args['image_key'])),
5     ScaleIntensityD(keys=(args['image_key'])),
6     SpacingD(KEYS, pixdim=(1., 1., 1.), mode=("bilinear", "nearest")),
7     OrientationD(KEYS, axcodes=args['axcodes']),
8     RandCropByPosNegLabelD(KEYS, label_key=args['mask_key'],
9                             spatial_size=(-1, -1, args['num_slices']),
10                            pos=args['pos_rand_crop'], neg=args['neg_rand_crop']
11                            ),
12     EnsureTyped(keys=KEYS, device=args['device']),
13     ResizeWithRatio(keys=KEYS, image_size=sam_image_dim, mode='trilinear'),
14     ResizeWithPadOrCropd(keys=KEYS, spatial_size=(sam_image_dim, sam_image_dim
15     , -1), mode='constant'),
16     RandFlipD(KEYS, prob=0.5, spatial_axis=0),
17     RandRotate90D(KEYS, prob=0.5, spatial_axes=(0, 1))
18 ]
19 train_transforms = Compose(transform_train)

```

تحلیل مرحله به مرحله:

جدول ۲: مراحل pipeline آموزش

#	Transform	کار
۱	LoadImageD	بارگذاری فایل‌های NIfTI از دیسک
۲	EnsureChannelFirstD	افزودن بُعد کانال در ابتدا
۳	NormalizeIntensityD	نرمال‌سازی z-score: $(x - \mu)/\sigma$
۴	ScaleIntensityD	مقیاس به [۰, ۱] بر اساس min/max
۵	SpacingD	resample به فاصله‌ی فیزیکی ۱mm ^۳
۶	OrientationD	تغییر جهت به استاندارد (مثلاً RAS)
۷	RandCropByPosNegLabelD	برش تصادفی به ابعاد (-1, -1, num_slices)
۸	EnsureTyped	تبدیل به tensor و انتقال به GPU/CPU
۹	ResizeWithRatio	تغییر اندازه با حفظ نسبت ابعاد
۱۰	ResizeWithPadOrCropd	pad/crop به مربع sam_dim × sam_dim
۱۱	RandFlipD	قرینه‌کردن تصادفی روی محور °
۱۲	RandRotate90D	چرخش تصادفی ۹۰ درجه در صفحه‌ی (0,1)

مثال روی داده‌ی MS: فرض کن ورودی یک MRI با مودالیتی FLAIR و ابعاد (181, 217, 181) باشد.

مرحله	شکل تصویر
Load + EnsureChannelFirst	(1, 181, 217, 181)
Normalize + Scale	(1, 181, 217, 181)
1mm → SpacingD	(1, 181, 217, 181)
RAS → OrientationD	ممکن است محورها جابه‌جا شوند
slice ۳۰ → RandCropByPosNegLabel	(1, 181, 217, 30)
EnsureTyped	tensor (1, 181, 217, 30)
۱۰۲۴ → ResizeWithRatio	(1, 1024, 1024, 30)
ResizeWithPadOrCrop	(1, 1024, 1024, 30)
RandFlip + RandRotate90	(1, 1024, 1024, 30)

نکته: در train، TransposeD وجود ندارد. یعنی خروجی به شکل (C, H, W, D) است. این احتمالاً چون در طول آموزش، درون خود مدل یک بُعد batch اضافه می‌شود.

۲.۴ بخش ۲-۳: transform_val_test

```

1 transform_val_test = [
2     LoadImageD(KEYS),
3     EnsureChannelFirstD(keys=KEYS),
4     NormalizeIntensityD(keys=(args['image_key'])),
5     ScaleIntensityD(keys=(args['image_key'])),
6     OrientationD(KEYS, axcodes=args['axcodes']),
7     EnsureTyped(keys=KEYS, device=args['device']),
8     ResizeWithRatio(keys=KEYS, image_size=sam_image_dim, mode='trilinear'),
9     ResizeWithPadOrCropD(keys=KEYS, spatial_size=(sam_image_dim, sam_image_dim
10     , -1), mode='constant'),
11     TransposeD(keys=KEYS, indices=(0, 3, 1, 2))
12 ]
13 test_val_transforms = Compose(transform_val_test)

```

تحلیل:

جدول ۳: مراحل pipeline ارزیابی

#	Transform	کار
۴-۱	مشابه train	بارگذاری، کانال، نرمال‌سازی، مقیاس
۵	OrientationD	بدون SpacingD
۶	EnsureTyped	تبدیل به tensor
۷	ResizeWithRatio	حفظ نسبت ابعاد
۸	ResizeWithPadOrCropd	pad/crop به مربع
۹	TransposeD(indices=(0,3,1,2))	جابه‌جایی محورها

تفاوت‌های کلیدی با train:

- بدون SpacingD: چون در val/test انتظار می‌رود داده‌ها از قبل پردازش شده باشند.
- بدون RandCropByPosNegLabelD: چون می‌خواهیم کل تصویر را ارزیابی کنیم، نه یک برش تصادفی.

- بدون augmentation: چون در ارزیابی نباید داده را تغییر داد.
- دارای TransposeD: در انتها محورها را جابه‌جا می‌کند.

۱.۲.۴ بررسی TransposeD(indices=(0, 3, 1, 2))

اگر ورودی (C, H, W, D) = (1, 1024, 1024, 30) باشد:

مقدار	محور جدید	معادل قدیمی
C = 1	۰	۰
D = 30	۳	۱
H = 1024	۱	۲
W = 1024	۲	۳

نتیجه: (C, D, H, W) = (1, 30, 1024, 1024).

چرا؟ چون SAM روی تصاویر 2D کار می‌کند. با این transpose، بُعد D (تعداد slice) به جایگاه بُعد batch می‌رود و هر slice به صورت جداگانه به SAM داده می‌شود. به عبارت دیگر، مدل ۳۰ تصویر 2D با ابعاد (1, 1024, 1024) می‌بیند.

۵ مقایسه‌ی train و val/test

جدول ۴: مقایسه‌ی گام‌های train و val/test

Val/Test	Train	Transform
✓	✓	LoadImageD
✓	✓	EnsureChannelFirstD
✓	✓	NormalizeIntensityD
✓	✓	ScaleIntensityD
×	✓	SpacingD
✓	✓	OrientationD
×	✓	RandCropByPosNegLabelD
✓	✓	EnsureTyped
✓	✓	ResizeWithRatio
✓	✓	ResizeWithPadOrCropd
×	✓	RandFlipD
×	✓	RandRotate90D
✓	×	TransposeD

۶ ارتباط با پروژه‌ی Polyp

این فایل مخصوص MS (سه‌بعدی و چندمودالیتی) نوشته شده و برای polyp باید تغییرات اساسی بدهد:

جدول ۵: مقایسه‌ی نیازمندی‌های MS و Polyp

ویژگی	MS (فعلی)	Polyp (هدف)
ابعاد داده	3D حجم MRI	2D تصویر RGB
مودالیتی	چندگانه (flair, t1, t2)	تک (RGB)
فرمت	NIfTI	PNG
SpacingD	✓ (معنای فیزیکی mm)	✗ (بی‌معنا)
OrientationD	✓ (آناتومی مغز)	✗ (بی‌معنا)
RandCropByPosNegLabelD	✓ (ضایعه کوچک)	بسته به تصمیم
TransposeD	✓ (تبدیل 3D به 2D)	✗ (از ابتدا 2D)
ResizeWithRatio	حفظ نسبت برای حجم 3D	قابل نگاه‌داشتن (مفید)

نکات مهم برای انتقال به polyp:

- LoadImageD، EnsureChannelFirstD، NormalizeIntensityD و ScaleIntensityD قابل استفاده‌اند.
- SpacingD و OrientationD باید حذف شوند (مخصوص داده‌های پزشکی حجمی هستند).
- RandCropByPosNegLabelD می‌تواند حفظ شود ولی با تنظیمات متفاوت (مثلاً اندازه‌ی crop برابر کل تصویر).
- TransposeD حذف می‌شود.
- ResizeWithRatio و ResizeWithPadOrCropd مفید می‌مانند (برای استانداردسازی ابعاد ورودی SAM).

۷ جمع‌بندی

این فایل، **pipeline پردازش داده‌ی MS** را تعریف می‌کند:

۱. بارگذاری حجم MRI چندمودالیتی از فایل NIfTI.

۲. پیش‌پردازش: نرمال‌سازی، resample به 1mm، جهت‌گیری آناتومیک.

۳. برش تصادفی به تعداد num_slices برش.

۴. آماده‌سازی برای SAM: تغییر اندازه به 1024×1024 با حفظ نسبت، pad/crop.

۵. Augmentation در train (rotate و flip).

۶. **Transpose** نهایی در val/test برای تبدیل (C, H, W, D) به (C, D, H, W).

برای polyp، این pipeline به شدت ساده‌تر می‌شود چون داده‌ها 2D و RGB هستند و بخش زیادی از این transformها (crop، orientation، spacing، transpose) اصلاً کاربردی ندارند.